

Entregável 6 — Cenários de verificação dos RNFs

[RNF01] — Segurança de autenticação

Categoria: Segurança.

Descrição: O sistema deve garantir que as senhas dos usuários não sejam armazenadas em texto puro e que as rotas autenticadas sejam acessíveis somente mediante um token JWT válido.

Componente/processo afetado: Back-end, especialmente `app/core/security.py` e `deps.py`.

Condições iniciais: Banco de dados populado com usuários de demonstração.

Procedimento: Verificar no banco se as senhas estão armazenadas como hash. Em seguida, acessar uma rota protegida sem token e depois utilizando um token inválido.

Métrica: Senhas armazenadas utilizando hash `bcrypt/passlib` e retorno HTTP 401 para requisições sem autenticação válida.

Resultado esperado: Nenhuma senha deve estar armazenada em texto puro e acessos sem JWT válido devem ser recusados.

Evidência: Captura de tela do banco mostrando o campo de senha em formato de hash e captura da resposta HTTP 401 no Swagger ou Postman.

[RNF02] — Confidencialidade e controle de acesso

Categoria: Segurança / Confidencialidade.

Descrição: Um aluno não pode visualizar informações pertencentes a outro aluno e um Personal Trainer somente pode acessar alunos vinculados a ele.

Componente/processo afetado:

`StudentService.garantir_acesso_aluno_por_perfil`, utilizado pelos serviços de treino, evolução e relatório.

Condições iniciais: Existência de dois alunos diferentes e usuários autenticados com perfis distintos.

Procedimento: Autenticar como Aluno A e tentar consultar os dados de evolução do Aluno B.

Métrica: Retorno HTTP 403 Forbidden.

Resultado esperado: O sistema deve bloquear o acesso não autorizado.

Evidência: Captura da resposta 403 no Swagger ou Postman.

[RNF03] — Tempo de resposta

Categoria: Desempenho.

Descrição: A consulta dos indicadores de evolução deve apresentar resposta em até 3 segundos no ambiente de demonstração.

Componente/processo afetado: Endpoint de consulta dos indicadores de evolução.

Condições iniciais: Banco contendo aproximadamente seis meses de histórico de evolução por aluno.

Procedimento: Realizar cinco consultas consecutivas aos indicadores de evolução e medir o tempo de resposta.

Métrica: Tempo de resposta ≤ 3 segundos em todas as cinco consultas.

Resultado esperado: Todas as consultas devem respeitar o limite estabelecido.

Evidência: Captura da aba Network do navegador ou resultado do comando de medição de tempo.

[RNF04] — Usabilidade e responsividade

Categoria: Usabilidade.

Descrição: A aplicação deve funcionar adequadamente em computadores e dispositivos móveis, apresentando mensagens claras para erros e operações realizadas.

Componente/processo afetado: Front-end React + Tailwind.

Condições iniciais: Sistema executando em localhost:5173.

Procedimento: Acessar os dashboards do Personal Trainer e do Aluno em resolução desktop de 1440px e mobile de 375px. Realizar também um login com senha incorreta.

Métrica: Não deve haver elementos cortados ou sobrepostos e as mensagens devem ser compreensíveis.

Resultado esperado: Interface responsiva e mensagem de erro clara, como “Credenciais inválidas”.

Evidência: Capturas das duas resoluções e da mensagem de erro.

[RNF05] — Portabilidade

Categoria: Portabilidade.

Descrição: O TrainUp deve ser executado utilizando os ambientes nativos de Python, Node.js e PostgreSQL, sem dependência de Docker.

Componente/processo afetado: requirements.txt, package.json e README.

Condições iniciais: Máquina com Python 3.11+, Node.js 18+ e PostgreSQL 14+.

Procedimento: Executar o back-end e front-end seguindo as instruções do README, sem utilizar Docker.

Métrica: Back-end e front-end iniciados corretamente.

Resultado esperado: API disponível em localhost:8000/docs e aplicação web em localhost:5173.

Evidência: Captura dos dois terminais funcionando simultaneamente.

[RNF06] — Disponibilidade em nuvem

Categoria: Disponibilidade / Nuvem.

Descrição: O sistema deve estar preparado para implantação em nuvem

utilizando variáveis de ambiente, sem credenciais diretamente no código.

Componente/processo afetado: render.yaml e app/core/config.py.

Condições iniciais: Repositório disponível no GitHub.

Procedimento: Verificar o código para garantir que não existem senhas ou strings de conexão fixas e realizar ou simular a implantação utilizando o Render.

Métrica: Ausência de credenciais hardcoded e possibilidade de acesso público após implantação.

Resultado esperado: Aplicação preparada para implantação em nuvem e capaz de disponibilizar o front-end conectado ao back-end.

Evidência: Captura do painel do Render ou do arquivo .env.example e URL pública, quando disponível.

[RNF07] — Confiabilidade dos dados de evolução

Categoria: Confiabilidade.

Descrição: Um novo registro de evolução não pode sobrescrever registros anteriores.

Componente/processo afetado: EvolutionService.registrar_evolucao e tabela evolucoes.

Condições iniciais: Aluno com pelo menos um registro de evolução cadastrado.

Procedimento: Registrar uma nova evolução em uma data diferente e consultar novamente todo o histórico.

Métrica: A quantidade de registros deve aumentar em um e o registro anterior deve permanecer inalterado.

Resultado esperado: O histórico deve manter todos os registros anteriores e acrescentar o novo registro.

Evidência: Capturas do histórico antes e depois do cadastro.

[RNF08] — Manutenibilidade e arquitetura em camadas

Categoria: Manutenibilidade.

Descrição: O back-end deve separar as responsabilidades entre API, regras de negócio e acesso aos dados.

Componente/processo afetado: backend/app/api, services, repositories e models.

Condições iniciais: Código-fonte disponível para análise.

Procedimento: Analisar o fluxo de registro de evolução, verificando a separação entre rota, serviço e repositório.

Métrica: Ausência de lógica de negócio e consultas SQL diretas dentro das rotas da API.

Resultado esperado: A rota deve delegar a operação ao EvolutionService, que utiliza o repositório responsável pelo acesso aos dados.

Evidência: Capturas ou trechos dos arquivos de rota, service e repository.

Entregável 7 — Diagrama de Casos de Uso

O diagrama terá como **atores principais** o **Personal Trainer** e o **Aluno**, dentro do sistema **TrainUp**. O guia determina que sejam representados os 16 RFs e que sejam incluídas as relações entre “Solicitar Agendamento” → “Definir Disponibilidade” e “Gerar Relatório” → “Consultar Evolução”.

Casos de uso

Personal Trainer:

- Registrar-se
- Login
- Cadastrar Aluno
- Editar Aluno
- Criar Treino
- Consultar Treinos
- Registrar Evolução
- Consultar Evolução/Indicadores
- Gerar Relatório
- Consultar Relatórios
- Gerenciar Serviços
- Definir Disponibilidade
- Confirmar/Rejeitar Agendamento
- Cancelar Agendamento
- Consultar Dashboard

Aluno:

- Registrar-se
- Login
- Consultar Treinos
- Consultar Evolução/Indicadores
- Consultar Relatórios

- Consultar Serviços
- Solicitar Agendamento
- Cancelar Agendamento
- Consultar Dashboard

